

D2-UD1



Las tecnologías **Java Servlets, JSP, JSF, J2EE y EJB** forman parte del ecosistema Java para el desarrollo de aplicaciones web y empresariales.

Originalmente se hablaba de J2EE (Java 2 Platform, Enterprise Edition) y posteriormente de Java EE (Java Platform, Enterprise Edition). Desde que el proyecto pasó a la Eclipse Foundation, el nombre actual es Jakarta EE.

Aunque todas están relacionadas con Java, cada una tiene una función diferente, pero la idea general es desarrollar aplicaciones que se ejecutan en el servidor mientras que el usuario accede a ellas mediante un navegador.

JAVA SERVLETS

Un Servlet es una clase escrita en Java que se ejecuta en un servidor y permite procesar las peticiones realizadas por los usuarios. Su función principal es gestionar peticiones HTTP y generar respuestas dinámicas que posteriormente se envían al navegador. De forma simplificada, el funcionamiento puede representarse como:



navegador → petición HTTP → Servlet → procesamiento → respuesta HTTP → navegador.

Normalmente, un Servlet extiende de la clase `HttpServlet` y puede encargarse de diferentes tipos de peticiones, como las peticiones GET y POST, además de operaciones relacionadas con la actualización o eliminación de información.

LENGUAJE DE PROGRAMACIÓN

Los Servlets utilizan Java como lenguaje de programación. En las versiones actuales de Jakarta EE se utilizan principalmente los paquetes `jakarta.servlet.*`, mientras que en versiones anteriores se utilizaban los paquetes `javax.servlet.*`.

REQUISITOS DE DESARROLLO

Para desarrollar una aplicación basada en Servlets es necesario disponer de un JDK (Java Development Kit), un IDE, un gestor de dependencias como Maven o Gradle y la API de Jakarta Servlet.

REQUISITOS DE EJECUCIÓN

Para ejecutar un Servlet se necesita un contenedor de Servlets o un servidor de aplicaciones compatible con Jakarta EE, como Apache Tomcat, Eclipse GlassFish o Payara.

El contenedor es el encargado de gestionar el ciclo de vida del Servlet. Entre sus funciones se encuentran cargar el Servlet, crear su instancia, inicializarlo, atender las peticiones recibidas y, finalmente, destruirlo cuando deja de ser necesario.

COMPATIBILIDAD CON LOS SERVIDORES WEB

Los Servlets no pueden ejecutarse simplemente abriendo un archivo desde el navegador. Necesitan un contenedor de Servlets que reciba las peticiones HTTP y se encargue de ejecutar el código Java correspondiente.

El proceso puede representarse de la siguiente manera:

Navegador → Servidor web/servidor de aplicaciones → Contenedor Servlet → HttpServlet → Respuesta HTTP

El contenedor de Servlets se encarga, entre otras funciones, de recibir las peticiones HTTP, asociar las URL con los Servlets correspondientes, ejecutarlos y gestionar su ciclo de vida. También puede encargarse de aspectos como las sesiones y determinados mecanismos de seguridad, antes de devolver la respuesta al cliente.

IDES UTILIZADOS

Para desarrollar aplicaciones basadas en Java y Jakarta EE pueden utilizarse diferentes entornos de desarrollo. Algunos de los más habituales son Eclipse, Apache NetBeans e IntelliJ IDEA.

EJEMPLO DE APLICACIÓN

Un ejemplo sencillo sería un Servlet que responde cuando el usuario accede a la dirección /hola:

```
@WebServlet("/hola")

public class HolaServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws IOException {

        response.setContentType("text/html");

        response.getWriter().println(

            "<html>" +

            "<body>" +

            "<h1>Hola mundo</h1>" +

            "</body>" +

            "</html>"

        );

    }

}
```

Si accedemos a <http://localhost:8080/miapp/hola>, el Servlet genera una página HTML y la devuelve al navegador. La anotación `@WebServlet("/hola")` indica la URL que estará asociada a este Servlet.

PÁGINAS WEB PÚBLICAS REALIZADAS CON ESTA TECNOLOGÍA

Algunos ejemplos de sitios en los que actualmente se detecta el uso de Java Servlets son la Junta de Andalucía, Southwire, Lenovo y Bpost.

Por ejemplo, la **Junta de Andalucía** cuenta con diferentes servicios y trámites online que necesitan procesar las peticiones de los usuarios y generar respuestas de forma dinámica. En este tipo de aplicaciones, los Servlets pueden encargarse de recibir las peticiones realizadas desde el navegador, procesarlas en el servidor y devolver la información correspondiente.

En el caso de **Lenovo**, los Servlets pueden formar parte de aplicaciones web que gestionan información de productos, servicios y diferentes operaciones realizadas por los usuarios. **Southwire**, empresa dedicada a productos y soluciones relacionadas con el sector eléctrico, también utiliza tecnología Java en parte de sus servicios web. Por otro lado, **Bpost**, el servicio postal de Bélgica, dispone de diferentes servicios online en los que es necesario procesar las peticiones de los usuarios y gestionar información desde el servidor.

JAKARTA SERVER PAGES (JSP)

Jakarta Server Pages (JSP) es una tecnología orientada a la creación de páginas web dinámicas mediante una combinación de HTML y elementos de Java/Jakarta. Su principal ventaja frente a escribir todo el contenido directamente en un Servlet es que permite trabajar de una forma más cómoda con páginas que tienen una estructura similar a HTML.



JSP está estrechamente relacionada con los Servlets, ya que una página JSP es procesada por el servidor y finalmente traducida y compilada como un Servlet. De esta forma, aunque el desarrollador trabaje inicialmente con una página JSP, el servidor utiliza internamente la tecnología de Servlets para procesarla y generar la respuesta.

LENGUAJE DE PROGRAMACIÓN

JSP utiliza principalmente HTML/XHTML, junto con elementos de Java/Jakarta, Expression Language (EL), etiquetas JSP y JSTL (Jakarta Standard Tag Library).

Aunque JSP permite introducir código Java directamente dentro de las páginas, actualmente se recomienda separar la lógica de negocio de la parte visual. De esta manera, JSP se utiliza principalmente para representar la información que se muestra al usuario.

REQUISITOS PARA DESARROLLAR

Para desarrollar aplicaciones utilizando JSP se necesita un JDK, un IDE y un servidor o contenedor compatible con JSP y Servlets. También es necesario disponer de la API de Jakarta Server Pages y, normalmente, utilizar un gestor de dependencias como Maven o Gradle.

REQUISITOS PARA EJECUTAR

Una página JSP no se ejecuta directamente en el navegador. Para poder utilizarla es necesario disponer de un servidor que proporcione un contenedor JSP/Servlet.

El funcionamiento puede representarse de la siguiente manera:

Página JSP → Contenedor JSP → Servlet generado → HTML → Navegador

El contenedor procesa la página JSP y la transforma en un Servlet, que posteriormente genera el contenido HTML que finalmente recibe el navegador.

COMPATIBILIDAD CON SERVIDORES WEB

JSP está estrechamente relacionada con los Servlets, por lo que necesita un entorno compatible con ambas tecnologías. Algunos ejemplos son Apache Tomcat, Eclipse GlassFish y Payara, además de otros servidores de aplicaciones compatibles con Jakarta EE.

IDES UTILIZADOS

Al igual que ocurre con los Servlets, para desarrollar aplicaciones JSP pueden utilizarse diferentes entornos de desarrollo, entre ellos Eclipse, Apache NetBeans e IntelliJ IDEA.

EJEMPLO DE APLICACIÓN

```
<!DOCTYPE html>

<html>

<head>

  <title>Ejemplo JSP</title>

</head>

<body>

  <h1>Mi página JSP</h1>

  <p>Hola ${nombre}</p>

</body>

</html>
```

En este ejemplo, la expresión **\${nombre}** utiliza **Expression Language (EL)** para obtener un valor que puede haber sido proporcionado por el servidor y mostrarlo dentro de la página. De esta forma, el contenido de la página puede cambiar dinámicamente dependiendo de los datos recibidos.

PÁGINAS WEB PÚBLICAS REALIZADAS CON ESTA TECNOLOGÍA

Podemos encontrar ejemplos como los servicios web del **Instituto Nacional de la Seguridad Social de Italia**, el portal de **EPFO** de India, **SUNAT** de Perú e **Interac** de Canadá.

Por ejemplo, el **INPS** utiliza servicios web para que los ciudadanos puedan consultar información y realizar diferentes trámites relacionados con la seguridad social. El portal de **EPFO** permite a los usuarios realizar consultas y gestionar diferentes servicios relacionados con la seguridad social y los fondos de pensiones en India. Por su parte, **SUNAT** ofrece servicios online relacionados con la administración tributaria de Perú, mientras que **Interac** proporciona servicios financieros y de pago digitales en Canadá.

JAKARTA SERVER FACES

Jakarta Server Faces, conocido originalmente como **JavaServer Faces (JSF)**, es un framework de Jakarta EE destinado al desarrollo de interfaces de usuario para aplicaciones web. Facilita la creación de aplicaciones mediante un modelo basado en componentes reutilizables.



Permite trabajar con elementos como formularios, botones, campos de texto, tablas y mensajes, relacionándolos con la lógica de la aplicación. También permite gestionar el estado de la interfaz, controlar eventos, validar y convertir datos y gestionar la navegación entre páginas.

Jakarta Faces sigue el patrón **MVC (Modelo-Vista-Controlador)** y está integrado con otras tecnologías de Jakarta EE. Actualmente, la versión estable más reciente es **Jakarta Faces 4.1**, incluida en **Jakarta EE 11**. La siguiente versión, Jakarta Faces 5.0, se encuentra en desarrollo como parte de Jakarta EE 12.

Su funcionamiento se basa en componentes: cuando el usuario realiza una petición, Jakarta Faces la procesa y genera la respuesta correspondiente. Entre sus características destacan la **gestión del estado**, los eventos, la **validación y conversión de datos**, la navegación entre páginas, los mensajes de error y el soporte para diferentes idiomas y configuraciones regionales.

LENGUAJE DE PROGRAMACIÓN

El lenguaje principal utilizado en Jakarta Faces es **Java**, ya que forma parte del ecosistema Jakarta EE. La lógica de la aplicación se desarrolla mediante clases Java que procesan información y responden a las acciones del usuario.

Para construir las interfaces se utilizan principalmente **Facelets**, una tecnología de vistas basada en **XHTML**. También se emplean HTML, CSS y JavaScript cuando son necesarios.

Jakarta Faces proporciona componentes y mecanismos que permiten relacionar los elementos visuales con las clases Java de la aplicación. Además, puede utilizar **Expression Language (EL)** para acceder a propiedades de objetos Java desde las páginas.

REQUISITOS PARA DESARROLLAR

Para desarrollar una aplicación utilizando Jakarta Faces se necesita un entorno Java compatible con la versión de Jakarta EE utilizada. En el caso de **Jakarta Faces 4.1**, el **requisito mínimo es Java SE 17**.

También es habitual utilizar **Apache Maven** para gestionar las dependencias, compilar la aplicación y preparar su despliegue.

Para desarrollar una aplicación se puede contar con:

- Java JDK 17 o superior.
- Un IDE compatible con Java y Jakarta EE.
- Apache Maven u otra herramienta de construcción.
- Las dependencias de Jakarta Faces.
- Un servidor o runtime compatible con Jakarta EE.

REQUISITOS PARA EJECUTAR

Una aplicación Jakarta Faces necesita ejecutarse en un **servidor compatible con Jakarta EE**. No se puede ejecutar simplemente abriendo el archivo XHTML desde un navegador, porque las páginas necesitan ser procesadas por el servidor.

El servidor recibe las peticiones HTTP del navegador, ejecuta la lógica correspondiente de la aplicación y genera la respuesta que finalmente recibe el usuario.

En una aplicación típica, el navegador solicita una página, el servidor procesa la petición mediante Jakarta Faces y las demás tecnologías utilizadas, se ejecuta la lógica Java necesaria y finalmente se genera una respuesta HTML que el navegador puede mostrar.

Para una aplicación basada en Jakarta Faces 4.1 se requiere además un entorno compatible con Java SE 17 o superior.

COMPATIBILIDAD CON SERVIDORES WEB

Jakarta Faces está pensado para ejecutarse dentro del ecosistema **Jakarta EE**, por lo que su compatibilidad depende del servidor o runtime utilizado y de la versión de Jakarta EE que soporte.

Para Jakarta EE 11 existen diferentes productos compatibles, entre ellos **Eclipse GlassFish, Payara Community, Payara, Open Liberty e IBM WebSphere Liberty**.

También existen aplicaciones Jakarta Faces que pueden ejecutarse utilizando otros servidores, como WildFly u Open Liberty, aunque la compatibilidad depende de la versión de Jakarta EE y de las dependencias utilizadas.

No se debe confundir Jakarta Faces con un servidor web: **Jakarta Faces es el framework utilizado para desarrollar la aplicación, mientras que el servidor proporciona el entorno necesario para ejecutarla.**

IDES UTILIZADOS

Jakarta Faces puede desarrollarse utilizando diferentes **IDE**. Entre los más habituales se encuentran **Eclipse, Apache NetBeans e IntelliJ IDEA**.

Eclipse permite trabajar con proyectos Jakarta EE y configurar servidores de aplicaciones. Apache NetBeans también permite desarrollar aplicaciones Java web y trabajar con tecnologías de Jakarta EE.

En **IntelliJ IDEA** existe un plugin llamado **Jakarta EE: Server Faces (JSF)** que proporciona soporte para desarrollar con Jakarta Faces.

La elección del IDE no cambia el funcionamiento de Jakarta Faces; simplemente proporciona herramientas para desarrollar y ejecutar el proyecto.

EJEMPLO DE APLICACIÓN

```
<!DOCTYPE html>

<html xmlns:h="jakarta.faces.html">

  <h:head> <title>Ejemplo Jakarta Faces

    </title>

  </h:head>

  <h:body>

    <h1>Mi página Jakarta Faces</h1>

    <h:form>

      <h:inputText value="#{usuario.nombre}" />

      <h:commandButton value="Saludar" action="#{usuario.saludar}" />

    </h:form>

  </h:body>

</html>
```

En este ejemplo se utiliza **Jakarta Faces** para crear una página web con un campo de texto y un botón. El atributo `value="#{usuario.nombre}` utiliza **Expression Language (EL)** para relacionar el campo de la página con una propiedad de una clase Java. Al pulsar el botón, `action="#{usuario.saludar}` ejecuta un método de esa clase. De esta forma, la interfaz puede interactuar directamente con la lógica de la aplicación.

PÁGINAS WEB PÚBLICAS REALIZADAS CON ESTA TECNOLOGÍA

Aunque no siempre es posible saber qué tecnología utiliza una página web únicamente visitándola, existen aplicaciones públicas que indican expresamente el uso de Jakarta Faces.

Un ejemplo es **OmniFaces Showcase**, una página que muestra diferentes componentes y funcionalidades para aplicaciones Jakarta Faces. Actualmente utiliza **Mojarra 4.1.5**, una

También existe **Guess Number**, un ejemplo oficial de Jakarta EE que utiliza Jakarta Faces para crear una pequeña aplicación web en la que el usuario debe adivinar un número.

Estos ejemplos permiten comprobar de forma directa cómo se utiliza Jakarta Faces en aplicaciones web públicas.

J2EE (JAKARTA ENTERPRISE EDITION ANTIGUA JAVA ENTERPRISE EDITION)

LENGUAJE DE PROGRAMACIÓN

Jakarta EE (anteriormente conocido como **Java EE** o **J2EE**) no es un lenguaje de programación, sino un **conjunto de especificaciones y estándares empresariales** basados en **Java**. Define cómo interactúan la capa web, la capa de negocio y la base de datos.



REQUISITOS PARA DESARROLLAR

Para desarrollar aplicaciones con Jakarta EE en un equipo local se necesita:

- **JDK (Java Development Kit):**
 - **Versiones recomendadas:** JDK 11, 17 o 21.
- **Gestor de dependencias y construcción de proyectos:**
 - **Apache Maven (pom.xml):** Automatiza la descarga de las librerías oficiales de la especificación Jakarta EE (como jakarta.jakartaee-api) desde repositorios centrales, gestionando las fases de compilación, empaquetado y ejecución de pruebas.
 - **Gradle (build.gradle):** Opción alternativa basada en DSL (Groovy/Kotlin) que ofrece mayor rendimiento en compilaciones incrementales.
- **Configuración del sistema:** Variable de entorno JAVA_HOME apuntando a la instalación del JDK instalado, asegurando que los scripts del gestor de construcción y los servidores locales la detecten.

REQUISITOS PARA EJECUTAR

Las aplicaciones Jakarta EE requieren un entorno administrado que gestione hilos, conexiones y ciclo de vida de los componentes.

- **Java Runtime Environment (JRE / JDK):** Entorno de ejecución sobre el servidor de producción para interpretar y ejecutar el bytecode Java.
- **Entorno de Servidor (Contenedores y Servidores de Aplicaciones):**
 - **Contenedores de Servlets (Ligeros):** Entornos como **Apache Tomcat** o **Eclipse Jetty** que implementan solo el perfil web de la especificación.

- **Base de Datos Relacional (RDBMS) y Persistencia:**

- **Sistemas compatibles:** MySQL, PostgreSQL, Oracle Database, MariaDB o Microsoft SQL Server.
- **Gestión mediante JPA:** En lugar de escribir consultas SQL directas, el servidor abstrae las operaciones CRUD en la base de datos mapeando clases Java (Entidades) directamente a tablas mediante anotaciones como **@Entity**, **@Table** y **@Id**.

COMPATIBILIDAD CON SERVIDORES WEB

Jakarta EE se integra nativamente en topologías de red modernas mediante arquitecturas de proxy inverso.

- Nginx
- Apache HTTP Server

Nginx y Apache HTTP Server actúan como primera capa ante los clientes en Internet mediante un proxy inverso HTTP/HTTPS. Reciben el tráfico entrante, gestionan los certificados SSL/TLS, comprimen las respuestas y balancean la carga redirigiendo las peticiones internas hacia el servidor de aplicaciones Java (como Tomcat o WildFly en el puerto 8080).

IDES UTILIZADOS

Los IDEs para Jakarta EE facilitan la depuración en tiempo real, el despliegue automático en servidores locales y el autocompletado del modelo de componentes.

- **IntelliJ IDEA (Ultimate Edition):** La herramienta preferida a nivel profesional. Ofrece autocompletado avanzado para JPA/CDI, inspección de dependencias y despliegue automatizado directo (*hot-deploy*) hacia servidores como Tomcat, WildFly o Payara.
- **Eclipse IDE for Enterprise Java Developers:** Cuenta con el ecosistema de complementos *Eclipse Web Tools Platform*, muy utilizado en la administración pública y grandes corporaciones.
- **Apache NetBeans:** Destaca por su integración nativa fuera de caja con servidores locales, simplificando la configuración inicial para principiantes.
- **Visual Studio Code:** Mediante la extensión *Java Extension Pack* e integraciones para Jakarta EE / Maven, permite desarrollar microservicios ligeros con poca sobrecarga de memoria.

EJEMPLO DE APLICACIÓN

A continuación se detalla cómo se estructuran las capas de un servicio web moderno con Jakarta EE:

Configuración de la Ruta Base (RestApplication.java)

Define el contexto raíz de las peticiones REST. Al heredar de jakarta.ws.rs.core.Application, habilita el motor JAX-RS en la aplicación.

```
package com.ejemplo;  
  
import jakarta.ws.rs.ApplicationPath;  
  
import jakarta.ws.rs.core.Application;  
  
@ApplicationPath("/api")  
  
public class RestApplication extends Application {  
  
}
```

Endpoint REST / Controlador de Dominio (SaludoResource.java)

Define el recurso que procesa la petición HTTP concreta, convierte los datos y devuelve una respuesta estructurada.

```
package com.ejemplo.api;  
  
import jakarta.ws.rs.GET;  
  
import jakarta.ws.rs.Path;  
  
import jakarta.ws.rs.Produces;  
  
import jakarta.ws.rs.core.MediaType;  
  
@Path("/saludo")  
  
public class SaludoResource {  
  
    @GET  
  
    @Produces(MediaType.APPLICATION_JSON)  
  
    public String obtenerSaludo() {  
  
        return "{\"mensaje\": \"¡Hola mundo desde Jakarta EE!\"}";  
  
    }  
  
}
```

PÁGINAS WEB PÚBLICAS REALIZADAS CON ESTA TECNOLOGÍA

Jakarta EE / Java EE es el estándar indiscutible en entornos donde **la estabilidad, la tolerancia a fallos, la transaccionalidad estricta y la compatibilidad hacia atrás** son requisitos prioritarios sobre la velocidad de prototipado.

- **Sedes Electrónicas de la Agencia Tributaria**
- **Grandes Entidades (BBVA, Santander, CaixaBank)**
- **Plataformas de Reservas de vuelo (Amadeus)**

EJB (ENTERPRISE JAVA BEANS)

EJB es un modelo de componentes del lado del servidor y una especificación de Jakarta EE (antes J2EE, impulsado por Oracle) diseñado para desarrollar aplicaciones empresariales.



Permite a los programadores enfocarse en la lógica de negocio de la aplicación y se encarga de simplificar el desarrollo al gestionar aspectos complejos como la seguridad, las transacciones y la concurrencia. Además, al basarse en componentes distribuidos, ofrece un software modular reutilizable, fácil de mantener y escalable.

LENGUAJE DE PROGRAMACIÓN

EJB utiliza Java como lenguaje de programación y forma parte de entornos empresariales. Emplea anotaciones y paquetes que pertenecen a Jakarta EE para determinar el comportamiento de los componentes repartidos por el servidor.

REQUISITOS PARA DESARROLLAR

Para desarrollar aplicaciones con EJB se necesitan cubrir ciertos requisitos:

Entorno de Software:

- Java Development Kit (JDK): Versión 8 o superior (depende de la especificación de Jakarta EE / Java EE usado).
- Un IDE compatible (como IntelliJ o Eclipse IDE) con Jakarta EE y un servidor de aplicaciones (como WildFly o GlassFish).
- Herramientas como Maven o Gradle para manejar librerías.

Componentes y Código:

- **Librerías de la API:** Es necesario incluir la dependencia de la API de EJB (jakarta.ejb-api) dentro del proyecto.
- Se deben emplear anotaciones obligatorias que indiquen el comportamiento y clase implementada (como @Stateless, @Stateful o @Singleton) de cada tipo de Session Bean, además de usar @Inject o @EJB para conectar los componentes.
- Los componentes se deben de compilar y empaquetar en archivos con formato .jar o formato .ear.

REQUISITOS DE EJECUCIÓN

Para ejecutar aplicaciones web EJB se necesita poseer un servidor compatible con Jakarta EE (como WildFly, GlassFish o Payara) que incluya un contenedor EJB. También es necesario tener Java (JRE/JDK) en su versión 8 o superior, empaquetar la aplicación en formato .war o .ear, y una base de datos configurada en el servidor con un DataSource si fuera necesario.

COMPATIBILIDAD CON LOS SERVIDORES WEB

Los EJB no se pueden ejecutar solo abriendo el archivo en el navegador ni en servidores web básicos o en servlets ligeros ya que necesitan estrictamente un servidor de aplicaciones con Java EE / Jakarta EE (como WildFly / JBoss EAP, GlassFish, IBM WebSphere / Liberty u Oracle WebLogic Server) que incluya un contenedor de EJB dedicado.

IDES UTILIZADOS

Para el desarrollo de EJB se encuentran distintos entornos de desarrollo como IntelliJ IDEA Ultimate, Eclipse IDE (Enterprise), Apache NetBeans y Oracle JDeveloper.

EJEMPLO DE APLICACIÓN

Un ejemplo de aplicación con EJB es el siguiente modelo, compuesto de dos partes: el componente de negocio que almacena el estado (EJB) y el Servlet que gestiona la petición web.

//EJB (lógica del negocio):

```
import jakarta.ejb.Stateful;
```

```
@Stateful //Guarda las interacciones de un usuario.
```

```
public class Pagina {
```

```
    private int visitas = 0;
```

```
    public int visitasTotales() {
```

```
        visitas++;
```

```
        return visitas;
```

```
    }
```

```
}
```

//Servlet / Página Web:

```
import jakarta.inject.Inject;

import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.HttpServlet;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

import java.io.PrintWriter;

@WebServlet("/contador")

public class ContadorServlet extends HttpServlet {

    @Inject

    private ContadorBean contador; // Inyecta el EJB

    protected void doGet(HttpServletRequest request, HttpServletResponse response)

        throws IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        int total = contador.visitasTotales();

        out.println("<html><body>");

        out.println("<h1>Ejemplo sencillo de EJB</h1>");

        out.println("<p>Has visitado esta página: <b>" + total + "</b> veces.</p>");

        out.println("</body></html>");

    }

}
```

Al acceder a <http://localhost:8080/proyectoEJB/contador> se podrá observar una página web donde el EJB incrementará las visitas y devolverá el número de veces que el usuario ha visitado el sitio.

PÁGINAS WEB PÚBLICAS REALIZADAS CON ESTA TECNOLOGÍA

Actualmente no existen páginas web públicas donde utilicen tecnología EJB ya que esta es una especificación de backend.

No obstante, hay sitios web que emplean una infraestructura basada en EJB. Los que más destacan son las plataformas gubernamentales y de administración pública (como las sedes electrónicas del gobierno de España o Grants.gov), grandes corporaciones y portales logísticos (Automatic Data Processing y Staples), y sistemas de banca y reservas en línea (como InterContinental Hotels Group).

COMPARATIVA CON OTRAS TECNOLOGÍAS

Jakarta EE está especialmente orientado a aplicaciones empresariales grandes, mientras que otras tecnologías como Node.js o PHP tienen otros enfoques y ecosistemas.

Característica	Jakarta EE	PHP	ASP.NET Core	Node.js
Lenguaje	Java	PHP	C#	JavaScript/TypeScript
Servidor	GlassFish, WildFly, etc.	Apache/Nginx	Kestrel/IIS	Node.js
Orientación	Empresarial	Web general	Web/empresarial	Web/servicios
Tipado	Estático	Dinámico	Estático	Dinámico/estático con TS
Ecosistema	Empresarial	Web muy extendida	Microsoft/.NET	JavaScript
Curva inicial	Media/alta	Baja/media	Media	Baja/media
IDEs	Eclipse, IntelliJ, NetBeans	VS Code, PhpStorm	Visual Studio, VS Code	VS Code, WebStorm

CONCLUSIÓN FINAL

En conclusión, la creación y evolución de estas tecnologías supuso un avance muy importante en el desarrollo de aplicaciones web y empresariales. Gracias a ellas, fue posible aprovechar Java para crear aplicaciones más dinámicas, organizadas, seguras y fáciles de mantener, separando diferentes partes del desarrollo y facilitando el trabajo de los programadores.

En conjunto, estas tecnologías contribuyeron a incorporar nuevas funcionalidades y adaptarse a las necesidades que iban surgiendo con el crecimiento de Internet. La posibilidad de contar con tecnologías que trabajan de forma conjunta también facilitó la creación de aplicaciones más completas y escalables.